

2.2 TASKS IN FOUR ABILITY CATEGORIES

Another aspect of NUPA is defining the tasks that models need to handle. The tasks should have clear calculation rules. Furthermore, most practical numerical processing tasks should either fall within these tasks or can be easily transformed into some of them. Extracted from the primary and secondary education curricula, we propose 17 tasks in four ability categories and students who have completed the stage of education are expected to solve them. The complete task list is shown in Table 1 and we provide a more detailed discussion in Appendix A.1.2 and an example for each task in Appendix A.1.3. Below we discuss the rationales for including some tasks in detail.

- **Elementary arithmetic: addition, subtraction, multiplication, and division.** The most fundamental mathematical operations. For division, we consider three types of related operators: **True division**, **floor division** and **modulus**.
- **Comparison: max and min.** Understanding numbers on the concept of “order”.
- **Digit understanding:** When we care about a language model’s understanding, processing (and generation) of numbers, digit is a crucial concept, as numbers are not read and processed by the language model as a whole, but rather as a sequence of digits. We specially designed some digit-related tasks to test whether LLMs truly handle digits, including:
 - **Get digit:** Given a number and an integer i , return the i -th digit. This task is important when certain digits have special meanings in a number (such as a phone number or SSN).
 - **Length:** Return the total length (i.e., the number of digits) of a number.
 - **Count:** Count the times that a particular digit occurs in an integer.
 - **Digit compare:** Compare and return the larger (smaller) digits one by one.
 - **Digit add:** Perform the normal addition digit by digit but ignore any carrying. For example, $\text{digit_add}(12345, 34567) = 46802$. It can test a model’s understanding of *digit alignment* and its mastery of single-digit addition.
- **Conversion between representations:** Converting a number to two representations: **to float** and **to scientific notation**, as they are frequently used to present final results. These two tasks test whether models can understand the relationship between various numerical formats. In particular, since many tasks present answers as approximate values, we designed a “**significant digit**” (**sig. fig.**) task to evaluate a model’s ability to round long numbers to fixed-length significant digits.

The combination of representations and tasks ultimately results in a total of 41 meaningful pairs. Without confusion, we refer to each combination as a task. The tasks receive either one or two numbers as inputs and return a number as result, and the input numbers and results share the same representation for most tasks unless otherwise stated (refer to Appendix A.1.4). The remaining combinations are excluded due to being excessively complex, uncommon, inapplicable, or redundant with other tasks. For further details, see the discussion in Appendix A.1.5.

The difficulty of each task depends not only on the nature of the task itself but also on the *length of the numbers* to be processed — longer tasks involve longer inputs and outputs as well as more steps of internal operations. Therefore, we test on different problem lengths. For tasks that are inherently more difficult, we limit the size of the problem to 1-20 digits, and for easier tasks to 1-100 digits. (For which tasks are considered difficult or easy, please refer to the Appendix A.1.6.)

We generated 1,000 questions for each task and each length. Unlike some previous works that set the lengths of two numbers to be the same, in our tests, the length L of a question is determined by the longer of the two numbers, while the length of the shorter number follows a uniform distribution between $L/2$ and L . We implemented additional handling to ensure that generated problems do not result in overly simple, complex, or meaningless results. Some tasks are further split into a hard and an easy version. More details about generating the benchmark are provided in Appendix A.1.7.

2.3 METRICS ABOUT NUPA

Measuring the performance of NUPA benchmarks on these tasks is not trivial. “**Exact match**” accuracy is the golden standard of the performance where the answer is considered as correct when it exactly matches the groundtruth. However, a smoother and more detailed metric is useful to understand the behavior and capabilities of a model. Therefore, we also report

Generation:	425.925535321
Groundtruth:	31415.92653582
Exact match:	0
Digit match:	8 / (8 + 5) = 0.62
dlength:	3

Figure 1: An example of metrics.

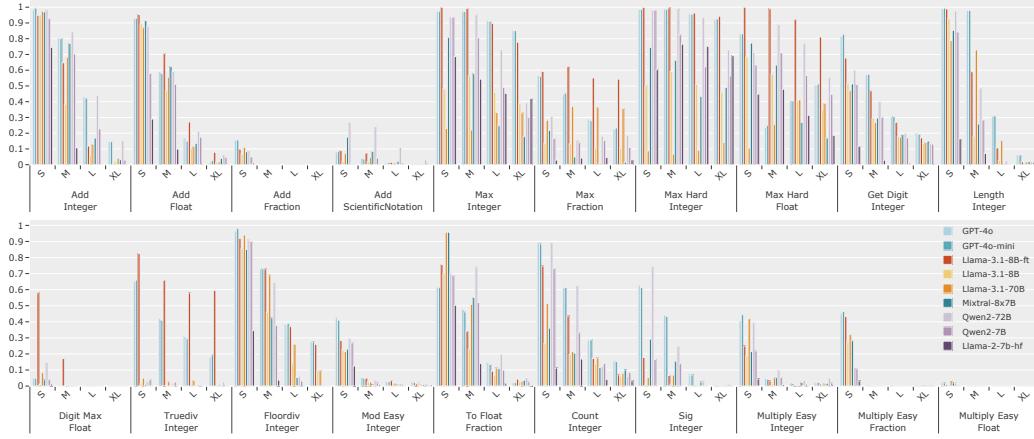


Figure 2: Parts of performance of state-of-the-art LLMs on NUPA benchmark.¹ “-ft” denotes a Llama model we finetuned on these tasks. (See Section 3.4)

the “**digit match**” and “**dlength**” (**difference of length**) metrics, as metrics of digit accuracy and length accuracy respectively. We first split numbers into parts (e.g., integer and decimal parts of a float, numerator and denominator of a fraction) and align the generated answer with the groundtruth digit by digit. Integer parts are aligned from the least significant digit; and the decimal parts of float are aligned from the most significant digit. For “digit match”, we measure the correctness of each digit, with missing digits considered as errors, and report the overall accuracy. For “dlength”, we report the sum of *absolute* difference in length between each part of the prediction and the groundtruth. Figure 1 illustrates these three metrics.

For each task, we divide the digits into four intervals (S, M, L, XL). For tasks with lengths 1-20, the four intervals correspond to 1-4, 5-8, 9-14, 15-20 digits respectively. For tasks with lengths 1-100, they correspond to 1-10, 11-20, 21-60, 61-100 digits respectively. We average the results in each interval for each task and metric. More details of our metrics are given in Appendix A.1.8

2.4 PERFORMANCE OF CURRENT LLMs

We test some commonly used LLMs on our benchmark, including three Llama models: Llama-2-7b, Llama-3.1-8b and Llama-3.1-70b (Meta, 2024a), one of the most popular open-source model families from Meta; Mistral-8×7B (MistralAI, 2024), a strong MoE model; and Qwen2-72B and Qwen2-7B (Qwen Team, 2024) which are also open-source models that are believed to have strong math abilities. Finally, we also test state-of-the-art commercial models GPT-4o-2024-08-06 and GPT-4o-mini-2024-07-18 (OpenAI, 2024a). We use prompts to control models to directly output result numbers without relying on external tools or CoT. The prompts used for each model and task are included in Appendix A.2. We select the results of some typical tasks in each category in Figure 2, while the complete results¹ and discussion on all metrics are shown in Appendix A.3. Here, we mainly focus on the zero-shot performance while we discuss few-shot performance in Appendix A.3.1. We have several observations regarding the results:

The best model performs well on typical tasks, but its performance declines on more specialized tasks. We find that GPT-4o, GPT-4o-mini and Qwen2 handle typical tasks, such as integer addition, float addition, integer max, and integer length, with high accuracy in the S and M ranges. This aligns with their strong performance on various mathematical datasets. However, their accuracy drops sharply when working with less common representations, like fractions and scientific notation, with average accuracy falling below 20%, even for the shortest S-range (1-4 digits). Similarly, for tasks such as significant figures, modulus operations, and digit-based calculations, their performance was unsatisfactory. This highlights the current limitations of LLMs in understanding numerical diversity and complexity. Despite their good performance on a narrow set of numerical tasks, they struggle with many others, failing to produce accurate results in these areas.

Length remains a significant challenge for NUPA of LLMs. We observe a noticeable decline in accuracy for even simple tasks like integer addition as the problem length increases. For instance, GPT-4o’s accuracy drops from nearly 100% in the S range and 80% in the M range to around 40% in the L range and just 15% in the XL range. In the more complex task float addition, the accuracy

¹An interactive performance report is shown in <https://huggingface.co/spaces/kangshijia/NUPA-Performance>.

decreases from 90% (S) and 60% (M) to merely 15% (L) and less than 5% (XL). This trend is consistent across other models and tasks. For example, Qwen2’s performance in the integer-length task declines from almost 100% in the S range to 50% in the M range, and falls below 5% in the L and XL ranges.

Length impedes learning both individual digits and overall length. To understand why models struggle with longer input numbers, we examine *digit match* and *dlength* performance in Figure 6 and Figure 7 in Appendix A.3. These metrics reveal that length affects **both** the accuracy of individual digits (digit match) and the answer’s overall length (dlength), with variations across tasks. For example, GPT-4o and Llama-3.1 display consistently low dlength in the add-integer task, with digit match decreasing sharply as length increases, suggesting that length primarily impacts per-digit accuracy on this task. Conversely, in the max-float task, dlength increases significantly with length (about 30-60 in the XL range), while digit match remains at 60% in the XL range. Note that since missing digits are treated as errors, this 0.6 digit match is likely due to these missing digits. This suggests that the main challenge here lies in generating answers of the correct length, rather than individual digit accuracy. In other tasks like fraction, both length and digit accuracy issues arise, as reflected in rising dlength and declining digit match.

“Digit” is more challenging than expected. We were surprised to find that LLMs struggle to fully grasp “digits”. For instance, in the “get digit” task, where the model is asked to return the i -th digit of a long integer, performance drops significantly as the length of the number increases. This suggests that current LLMs lack a consistent ability to simply find a digit. Note that the performance is good in the shorter S-range, which indicates that the models can at least comprehend the task instruction. In the XL-range, GPT-4o achieves only 20% accuracy, barely above the random guessing 10% baseline (since the correct answer is always a digit between 0 and 9). This fundamental limitation may explain why current LLMs struggle with numerical understanding and processing, especially as task complexity and input length increase. If a model cannot reliably identify a specific digit in a given number, it casts doubt on its ability to generalize to more complex arithmetic tasks, such as addition.

We also have some interesting observations: (1) LLMs find the “max-hard” task easier than “max” with integer inputs. The difference between the tasks is that in the max task, the two numbers often differ in length, whereas in max-hard, they are always the same length and share some left-most digits, requiring more digits to be compared. While max-hard intuitively seems more difficult, models actually perform better on it. This is likely because they struggle to effectively use sequence length information, as reflected in their weaker performance on the “length” tasks in the longer ranges. It suggests that models might process tasks in different ways from humans. They could have to compare two numbers digit by digit. In this situation, the “harder” subtasks are actually easier because the numbers are already aligned. (2) GPT-4o and GPT-4o-mini show nearly identical performance across most tasks, similar to the comparison between Qwen2-72B and Qwen2-7B. This suggests that once a model reaches a certain size, NUPA performance relies more on factors like architecture, training strategies, data diversity, and post-training refinements, rather than simply on increasing model size.

3 HOW DO TOKENIZERS, PES AND DATA FORMATS AFFECT NUPA?

We have observed that the NUPA Test poses significant challenges even for the most advanced LLMs. In this section, we aim to investigate the factors that can influence the NUPA of LLMs during their pretraining phase, including tokenization strategies, PES, and different data formats. We utilize the architecture of decoder-only transformers and alter the size to create models with 0.1B, 0.9B and 3B parameters. These models are trained from scratch, incorporating a wide range of techniques that could potentially impact NUPA. In this section, each model is trained on a *single* task. The details of the training process and models are included in Appendix A.4.1.

3.1 ONE-DIGIT TOKENIZERS ARE GOOD ENOUGH

LLMs interpret numbers as segmented tokens rather than whole numbers. With the development of language models, various tokenization strategies have emerged, including *mixed tokenizers*, *one-digit tokenizers*, and *k-digit tokenizers* ($k \geq 2$), as shown in Figure 3. In the BPE tokenizer used by GPT-2, the numbers are not specially treated, which resulted in irregular number cutting and is harmful to digit alignment. The cutting

(a) 31415.926535897932

(b) 31415.926535897932

(c) 31415.926535897932

(d) 31415.926535897932

Figure 3: Different tokenization of a long number. (a) GPT2: mixed digit tokenizer, (b) Llama-2: one-digit tokenizer. (c) GPT-3.5, GPT-4 and Llama-3: three-digit tokenizer. (d) Aligned three-digit tokenizer.